Stonks
Conceptual Interview Guide

100 practical questions and interview-ready answers

Designed for explaining the project clearly: how it works, why each technology is used, and how you would respond to common product and technical scenarios.

1. Project Overview and Architecture

1. What problem does Stonks solve?

Answer: Stonks is a stock-dashboard and portfolio-simulation application. It helps users view market data, track virtual holdings and cash, receive AI-assisted insights, read news, and explore ML-based predictions in one place.

2. Explain the project in one minute to an interviewer.

Answer: The frontend is built with Next.js and React. Next.js route handlers provide APIs for authentication, market data, portfolios, news, and AI chat. MySQL stores users and portfolio data; a FastAPI service handles Python ML models; Chroma supports semantic retrieval for the AI chatbot.

3. Why did you use Next.js?

Answer: It lets us build the UI and backend-for-frontend in the same project. It supports React-based pages, server APIs, routing, and production deployment without managing a separate Node backend at the beginning.

4. What are the main modules of the project?

Answer: The main modules are authentication, dashboard, market/stocks, portfolio and wallet, transactions, news, AI chat, buy-signal prediction, end-of-day prediction, and portfolio rating.

5. How do frontend and backend communicate?

Answer: React pages call Next.js API routes using HTTP. The routes validate requests, call the database or external providers/ML service, and return JSON used by the UI.

6. Why is the ML part separated into FastAPI?

Answer: The trained models and their Python libraries are easier to run in Python. Keeping them separate also prevents heavy model loading from slowing or complicating the Next.js server.

7. What would you improve if the app gets many users?

Answer: I would introduce shared caching for market data, rate limits, managed database connection pooling, queues for slow work, monitoring, and independently scale the ML service.

8. What is the role of environment variables?

Answer: They keep database credentials, JWT secrets, Firebase settings, provider keys, and ML URLs out of source code. Different environments can use different values without code changes.

9. Why are API routes useful instead of calling providers directly from the browser?

Answer: They hide API keys, centralize validation and error handling, apply caching and rate limits, and allow us to normalize data from different providers before the UI sees it.

10. If an interviewer asks about limitations, what would you say?

Answer: It is an educational/simulation-focused application. Market-data quality depends on providers, models cover limited tickers, and AI output is informative rather than financial advice.

2. UI, Dashboard and State

11. What does the dashboard show?

Answer: It acts as the user’s home screen for important information such as market movements, portfolio summary, quick navigation, and AI-driven features.

12. Why do some React components need to be client components?

Answer: Interactive features such as buttons, forms, charts, theme switching, and browser localStorage need client-side JavaScript. Non-interactive data rendering can stay server-side.

13. How is the wishlist handled?

Answer: The wishlist hook stores selected ticker symbols in browser localStorage. This is simple and fast for a personal UI preference, but it does not synchronize across devices.

14. What would you do if users request wishlist sync across devices?

Answer: Move it from localStorage to a user-owned database table, secure it with authentication, and load/save it through an API.

15. How would you handle a slow dashboard?

Answer: Load important summary data first, show loading/skeleton states, cache market data, avoid duplicate requests, and lazy-load less important charts or sections.

16. Why are loading and error states important?

Answer: External APIs and models can fail or be slow. Clear loading/error states prevent users from seeing a blank or misleading screen and tell them whether data is unavailable or simply delayed.

17. How would you make the interface accessible?

Answer: Use semantic HTML, labels for inputs, keyboard navigation, visible focus states, sufficient contrast, meaningful chart alternatives, and screen-reader-friendly error messages.

18. What if a user refreshes a page while submitting a trade?

Answer: The backend should protect against duplicate trade execution using an idempotency key or stored request ID. The UI alone cannot guarantee this.

19. How do you keep API response shapes consistent?

Answer: Define clear JSON contracts, validate inputs and outputs, centralize shared types/schemas where possible, and add tests for important API responses.

20. How would you explain dark mode/theme support?

Answer: A theme provider manages the selected theme and applies the appropriate CSS class. The preference can be stored so the experience remains consistent after reload.

3. Authentication and User Security

21. How does a new user create an account?

Answer: The signup API validates name, email, and password; checks that email is not already used; hashes the password; creates the user; then issues authentication cookies.

22. Why should passwords never be stored directly?

Answer: If the database is exposed, plain passwords immediately compromise every account. Password hashes are one-way values, so the server compares a login password to the hash instead.

23. What is bcrypt doing in this project?

Answer: bcrypt hashes passwords using a deliberately expensive algorithm. This slows attackers trying large numbers of guessed passwords after a data leak.

24. What are access and refresh tokens?

Answer: The access token is short-lived and used to prove identity for normal requests. The refresh token lasts longer and is used to create a new access token without asking the user to sign in again.

25. Why use cookies for tokens?

Answer: HttpOnly cookies reduce the chance that browser JavaScript can steal tokens through XSS. They are sent automatically with requests, so the server can authenticate the user.

26. What happens during login?

Answer: The server finds the user by email, compares the supplied password using bcrypt, creates signed tokens, stores a hash of the refresh token, and sends auth cookies back to the browser.

27. How does the dashboard know a user is logged in?

Answer: The proxy checks authentication cookies before dashboard routes are served. Protected API routes also verify the access token before returning private data.

28. What if the user gives a wrong password?

Answer: The app returns a generic invalid-email-or-password response. This is safer than revealing whether the email exists.

29. What if a token expires while the user is active?

Answer: A refresh flow should verify the refresh token and issue a new access token. If refresh is invalid or expired, the user is sent to login.

30. How would you make login more secure later?

Answer: Add rate limiting, password rules, email verification, MFA, device/session management, audit logs, and immediate token revocation for suspicious activity.

4. Database, Wallet and Portfolio

31. What does MySQL store in Stonks?

Answer: It stores durable user information, password/refresh-token data, wallet balances, portfolio holdings, and transaction history.

32. What is the difference between wallet, holdings, and transactions?

Answer: Wallet is unused virtual cash. Holdings are current positions such as quantity and average price. Transactions are the historical BUY and SELL records that explain how the portfolio changed.

33. Explain a BUY transaction flow.

Answer: The API authenticates the user, validates symbol, quantity, and price, checks the wallet balance, subtracts the trade cost, creates or updates the holding, records the transaction, and commits all changes together.

34. Explain a SELL transaction flow.

Answer: The API confirms the user owns enough quantity, reduces or removes the holding, adds sale value to wallet cash, calculates realized P&L; using average cost, records the sale, and commits it.

35. Why is a database transaction used?

Answer: A trade changes several records. A transaction ensures they all succeed together or all roll back, so balance, holdings, and history cannot disagree.

36. Why do we lock wallet and holding rows during trading?

Answer: A lock prevents two requests from reading the same balance or quantity at the same time. Without it, simultaneous requests could spend the same cash twice or sell more shares than held.

37. What is average buy price?

Answer: It is the weighted average amount paid per share for a holding. Buying more shares at another price changes the average based on both quantities and costs.

38. What is realized P&L;?

Answer: It is the profit or loss that becomes final when shares are sold. For a sale, it is approximately sale price minus average cost, multiplied by sold quantity.

39. What is unrealized P&L;?

Answer: It is the current paper gain or loss on shares still held, based on their latest market value compared with their cost basis.

40. What would you add for a real brokerage product?

Answer: Broker order integration, server-side executable quotes, order states, partial fills, fees, settlements, compliance controls, audits, stronger decimal accounting, and regulatory requirements.

5. Market Data and News

41. Where does stock data come from?

Answer: The project integrates market-data providers such as Finnhub and Twelve Data through server-side helper modules and normalizes data for the rest of the app.

42. Why use more than one market-data provider?

Answer: Providers can have different coverage, quotas, latency, or outages. Multiple providers allow fallback, but the app must clearly identify data source and freshness.

43. What does real-time data mean in this project?

Answer: It means the app attempts to request current quotes. The exact delay depends on the provider plan and market conditions, so the UI should not claim instant exchange-level execution data unless guaranteed.

44. What should happen if a market provider is unavailable?

Answer: The route should fail gracefully: use recent cached data when safe, show its timestamp, use a backup provider if configured, and avoid displaying invented prices.

45. Why should the app show data freshness?

Answer: A price from several minutes ago may be useful for learning but risky for decisions. A timestamp and live/stale label let the user judge reliability.

46. What is API rate limiting?

Answer: External providers restrict how many requests can be made in a time window. The app should cache and share results to avoid hitting limits for every individual user request.

47. How can caching help market data?

Answer: A short-lived cache can serve the same quote to many users, reduce latency and provider cost, and refresh in the background when it becomes stale.

48. How is news useful in a stock dashboard?

Answer: News provides context for price moves and lets users explore company and market events. Sentiment is an additional signal, not proof that a stock will rise or fall.

49. What does news sentiment mean?

Answer: It estimates whether wording is positive, negative, or neutral. It captures text tone, but it can miss nuance, sarcasm, relevance, and the difference between sentiment and market impact.

50. What if news has misleading information?

Answer: The app should show source and URL, avoid treating any article as fact, add clear disclaimers, and ensure the AI does not overstate claims from a single item.

6. AI Chatbot and Chroma

51. What is the purpose of the AI chatbot?

Answer: It lets users ask natural-language questions about available stock data, news, and previous conversation context, making the dashboard easier to explore.

52. What is RAG in simple terms?

Answer: Retrieval-Augmented Generation means the app first finds relevant stored information, then gives that information to the language model so its answer is grounded in current project data.

53. What is Chroma used for?

Answer: Chroma is the vector database used to store and search semantic representations of stocks, news, chat turns, and portfolio snapshots.

54. Why not send every stock and news article to the AI model?

Answer: It would be slow, costly, and may exceed the model context limit. Retrieval chooses a smaller set of information that is likely relevant to the question.

55. What are embeddings?

Answer: Embeddings are numeric representations of text meaning. Similar meanings tend to have nearby vectors, allowing the app to retrieve related text even when wording differs.

56. How does the chatbot answer a question?

Answer: It validates the prompt, retrieves market/news/chat context, sends a structured prompt with that context to the configured model, stores the conversation turn, and returns the answer and metadata.

57. Why is chat history stored?

Answer: It allows continuity, so follow-up questions can use relevant earlier conversation. History needs limits, privacy controls, and a clear retention policy.

58. What if Chroma is unavailable?

Answer: The project has a fallback path. In production, a shared durable vector store is preferable because local or in-memory fallback data can disappear or differ between servers.

59. Why should the chatbot not give guaranteed investment advice?

Answer: Language models can be wrong, incomplete, or influenced by unreliable context. The chatbot should explain uncertainty, cite available sources, and present educational information rather than promises.

60. What is prompt injection, and how would you reduce it?

Answer: It is when untrusted content tries to manipulate model instructions, for example a news item saying “ignore rules.” Treat retrieved content as data, limit sensitive context, and enforce important rules in application code.

7. Machine Learning Features

61. What ML features are included?

Answer: The project includes end-of-day price prediction, a buy-signal predictor, portfolio rating, news sentiment analysis, and support for a chatbot model/provider.

62. What is the end-of-day predictor?

Answer: It estimates a future end-of-day price for supported tickers. It is a model-based estimate, not a guaranteed future price.

63. Why are only AAPL and TSLA supported for EOD prediction?

Answer: The deployed model artifacts and preprocessing pipelines were trained for those tickers. Limiting the API prevents the system from pretending it can accurately predict unsupported stocks.

64. What is a buy-signal predictor?

Answer: It turns trained-model output into a BUY or SELL-style signal with supporting predicted values. It should be treated as a research/learning indicator, not a command to trade.

65. How is a prediction different from a recommendation?

Answer: A prediction is an uncertain estimate from historical patterns. A recommendation includes suitability, risk, objectives, regulation, and accountability; this application should not imply that level of personalized advice.

66. What data can influence stock models?

Answer: Typical features include past prices, volume, technical indicators, and sometimes news-related signals. Only data available at the prediction time should be used.

67. Why does model accuracy not guarantee profit?

Answer: Markets change, incorrect predictions can be costly, and fees, slippage, position size, and risk management affect actual returns. A model can have decent accuracy but poor trading results.

68. What is overfitting?

Answer: Overfitting occurs when a model memorizes historical noise rather than learning patterns that generalize. It can look strong in training but perform poorly on unseen future data.

69. Why is a separate ML service useful operationally?

Answer: It allows Python packages and models to be deployed, monitored, restarted, and scaled independently from the web application.

70. What should happen if ML inference fails?

Answer: Return a clear unavailable/error state or a clearly labelled simple fallback where appropriate. Never silently present a fallback as if it were the trained-model result.

8. Scenarios and Problem Solving

71. Scenario: two BUY clicks happen quickly. What concern do you raise?

Answer: The trade might be duplicated. Disable repeated UI submission, but more importantly enforce an idempotency key and transaction-safe backend logic.

72. Scenario: a user has Rs.1,000 and sends two Rs.800 buy requests. How is this prevented?

Answer: The database transaction locks the wallet row. The first request updates it; the second waits, sees the new balance, and is rejected for insufficient funds.

73. Scenario: the quote API starts returning old prices. What do you do?

Answer: Compare provider timestamps, mark the result stale, show the last-known time, stop calling it live data, and use cache/backup provider only if it meets a defined freshness rule.

74. Scenario: Gemini is down during chat.

Answer: Catch the failure with a timeout, optionally try a configured provider fallback, show a friendly retry message if both fail, and log the provider failure for monitoring.

75. Scenario: the user asks the chatbot, “Should I invest my savings in one stock?”

Answer: The assistant should avoid personalized financial advice, explain concentration risk generally, encourage diversification and independent research, and state the answer is educational.

76. Scenario: a user says they saw a different price on another website.

Answer: Explain that quotes may differ by provider, exchange, currency, delay, and regular versus after-hours session. Show source and as-of time so the discrepancy can be investigated.

77. Scenario: users complain chat answers are slow.

Answer: Measure each stage: market/news fetch, embedding generation, Chroma retrieval, model inference, and storage. Then cache, reduce retrieval/context, stream output, or improve provider capacity based on evidence.

78. Scenario: one user accesses another user's chat session ID.

Answer: That is an authorization flaw. The backend must derive ownership from the verified user token and filter all chat data by owner, not trust a session ID from the browser.

79. Scenario: database write succeeds but browser loses connection.

Answer: The user may retry even though the trade already happened. An idempotency key lets the server return the earlier result instead of processing a second order.

80. Scenario: the app is deployed to Vercel but Python models do not work.

Answer: Run FastAPI as a separate deployed service, expose a protected internal URL such as PY_AI_SERVICE_URL, add health checks, and let Next.js forward prediction requests to it.

9. Testing, Deployment and Reliability

81. What would you unit test first?

Answer: Input validation, weighted-average price logic, P&L; calculations, token helpers, data normalization, and error-handling helpers because they are deterministic and high impact.

82. What integration tests are important?

Answer: Test signup/login against a test database, authenticated portfolio fetches, trade commits/rollbacks, and API interaction with a test ML service or realistic mock.

83. How would you test a BUY transaction?

Answer: Start with a known wallet balance, submit a valid trade, then verify the wallet decreased correctly, holding quantity/average price are correct, and a transaction row exists.

84. How would you test failure of a trade?

Answer: Try insufficient funds, invalid quantity, invalid price, overselling, and forced database failure. Verify no partial wallet or holding changes remain.

85. Why test external API failures?

Answer: Provider outages, rate limits, malformed responses, and slow calls are normal production situations. Testing them ensures the UI shows an honest fallback rather than crashing.

86. What is a health endpoint?

Answer: It is a small endpoint, such as FastAPI /health, that infrastructure can call to check whether a service is alive and ready to receive requests.

87. How would you deploy this project?

Answer: Deploy the Next.js app to a web platform such as Vercel, provision MySQL and Chroma/shared services, deploy FastAPI separately, and configure production environment variables securely.

88. What should be monitored?

Answer: Error rate, request latency, provider failures, quote freshness, login failures, database connection/lock issues, chat fallback rate, ML latency, and unusual transaction activity.

89. Why should logs avoid sensitive values?

Answer: Logs are widely accessible during support and may be retained. Do not log passwords, tokens, secret keys, or unnecessary personal and portfolio details.

90. What is graceful degradation?

Answer: When a non-critical dependency fails, the product remains useful with an honest reduced experience, for example cached quote data marked with its time instead of a broken dashboard.

10. Deep Dive and Reflection

91. What is the strongest technical part of this project?

Answer: It combines a realistic end-to-end product flow: secure user accounts, transactional portfolio simulation, external market/news data, Python ML, and AI retrieval in one coherent application.

92. What was the most challenging integration?

Answer: The AI and market-data path is challenging because it combines several unreliable external dependencies, timeouts, retrieval, model providers, and user-facing latency expectations.

93. How do you explain data flow for a chat answer?

Answer: User question goes to the Next.js chat API; it obtains relevant stock/news/history context from provider APIs and Chroma; the model produces an answer; the system stores the new turn and returns it to the UI.

94. How do you explain data flow for a trade?

Answer: User submits a trade form; the portfolio API verifies identity and input; MySQL transaction checks and locks current balance/holding; it updates wallet and position, writes history, commits, and returns the new result.

95. What security concern would you improve first?

Answer: Ensure every private route is consistently authenticated and authorized, especially user-owned chat/portfolio resources; then add rate limits and robust refresh-token session management.

96. What performance improvement would you make first?

Answer: Create a shared cached market-data service to avoid repeated provider calls. This directly improves dashboard and chatbot latency while reducing quota usage.

97. What AI improvement would you make first?

Answer: Add user-scoped retrieval filters and answer citations. That improves privacy, traceability, and user trust before adding more model complexity.

98. What database improvement would you make first?

Answer: Replace request-time schema setup with versioned migrations and confirm the most important unique/index constraints. This makes deployments predictable and protects data integrity.

99. How would you answer “what did you learn?”

Answer: I learned that building AI and ML features is not only about models; reliable product behavior needs security, clean data flow, database correctness, provider failure handling, clear UX, and honest communication of uncertainty.

100. How would you close a project interview discussion?

Answer: I would explain that Stonks demonstrates a full-stack approach to a financial-learning product, while being clear about its current boundaries and the practical next steps needed for production-grade scale and financial safety.